

Contents

Fake Job Postings — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
How This Guide Works (Read This First!)	3
We do NOT give you the full code	3
Visualizations — Two Modes	3
1. Exploratory charts (for YOU)	3
2. Explanatory charts (for CARLOS)	4
Your plotting toolkit	4
Before You Start — Google Colab Setup	4
Step 1 — Open a new Colab notebook	4
Step 2 — Connect to Google Drive	4
Step 3 — Create a project folder	4
Step 4 — Get the data	4
Step 5 — Test it	5
Colab tips	5
Class 1 — Data Cleaning	5
Your goal	5
Inputs	5
Outputs	5
Phase A — Explore the data first (15 min)	5
Phase B — Clean the jobs table (45 min)	6
Phase C — Make ONE chart for Carlos (15 min)	7
Common mistakes in Class 1	7
Self-check before Class 2	7
Class 2 — Encoding and Scaling	7
Your goal	7
Inputs	7
Outputs	8
Phase A — Explore the data first	8
Phase B — Encode and scale	8
Phase C — Make ONE chart for Carlos	9
Common mistakes in Class 2	9
Self-check before Class 3	9
Class 3 — Feature Engineering	9
Your goal	9
Inputs	9
Outputs	9
Phase A — Explore the data first	10
Phase B — Engineer the features	10
Phase C — Make ONE chart for Carlos	11
Common mistakes in Class 3	11
Self-check before Class 4	11
Class 4 — Feature Selection	11

Your goal	12
Inputs	12
Outputs	12
Phase A — Explore the data first	12
Phase B — Select features	12
Phase C — Make ONE chart for Carlos	13
Common mistakes in Class 4	13
Self-check before Class 5	13
Class 5 — Pipelines	13
Your goal	13
Inputs	13
Outputs	14
Phase A — Explore the data first	14
Phase B — Build the pipeline	14
Phase C — Make ONE chart for Carlos	15
Common mistakes in Class 5	15
Self-check before Class 6	16
Class 6 — End-to-End Lab (THE BIG ONE)	16
Your goal	16
Time	16
Outputs	16
Phase A — Explore the data first (10 min)	16
Phase B — Build the final dataset (70 min)	16
Phase C — Findings report for Carlos (10 min)	17
Grading rubric (100 points)	18
Submit	18
Tips for the whole module	18

Fake Job Postings — Module 3 Lab Guide

Scenario: Trust & safety. Detect fake job postings before they reach job seekers. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at a **job search website** (like Indeed or LinkedIn). Companies post job openings. Job seekers apply.

Every week: - **50,000 new job postings** are submitted to your platform. - About **200 are SCAMS**. They want to steal money, identity, or personal information.

The problem

When a fake posting reaches job seekers: - People send their CV with their full address, ID, phone number. - Some send “training fees” or “background-check fees” to scam companies. - Your platform brand suffers.

Carlos is worried. He calls your team into a meeting.

Your manager's request

Your manager, **Carlos** (Head of Trust & Safety), tells you:

“Every week 200 fake postings slip through our manual review. They steal money from desperate job seekers.

I need a model. Detect fake postings BEFORE we publish them. Then we automatically block or flag them for review.

Two warnings: 1. The catch: only **5%** of postings are fake. If you build a model that always says ‘real’, you get 95% accuracy and ZERO catches. **Accuracy is a trap here.** Use F1. 2. Do NOT block REAL jobs by mistake. Every wrong block is a real company that goes to our competitor.”

Your team's job for the next 2 weeks (Module 3)

Carlos sends you a CSV with **18,000 postings**. About 900 of them are fake.

Your job in Module 3: > **Turn this CSV into ONE clean file. The clean file will be used to train the fake-detector in Module 4.**

The clean file is called `fake_jobs_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the fraud detector. Carlos gets his blocker.
Module 5	Find groups of similar legitimate jobs (industry clusters). Also: find anomalies.
Module 7	Use the description text alone. Find “scam phrases”.

You use the **same fake-jobs dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

You will see:

1. **WHAT / WHY / EXPECTED** — explained in words
2. Sometimes a code skeleton with blanks (___)

Why no full code?

You will NOT learn if you copy-paste. **Rule:** Redo every copy-pasted step by yourself.

Visualizations — Two Modes

1. Exploratory charts (for YOU)

Fast, ugly. Goal: “What does this data look like?”

2. Explanatory charts (for CARLOS)

Clean, labeled, one clear message.

Your plotting toolkit

Plot	When to use it	Function name
Histogram	Shape of a numeric column	<code>plt.hist()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	Outliers	<code>sns.boxplot()</code>
Heatmap	Correlation between columns	<code>sns.heatmap(df.corr())</code>
Stacked bar	Compare fraud vs real	<code>df.groupby('fraudulent')['col'].mean()</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**”.
4. Name it `fake_jobs_module_3.ipynb`.

Step 2 — Connect to Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Step 3 — Create a project folder

```
import os
os.makedirs('/content/drive/MyDrive/fake_jobs_lab', exist_ok=True)
%cd /content/drive/MyDrive/fake_jobs_lab
```

Step 4 — Get the data

Option A — Direct from Kaggle:

1. Free Kaggle account.
2. Upload `kaggle.json`:

```
from google.colab import files
files.upload()
```

3. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d shivamb/real-or-fake-fake-jobposting-prediction
!unzip -q real-or-fake-fake-jobposting-prediction.zip -d data
!ls data/
```

Option B — Upload by hand in Colab’s file panel.

Step 5 — Test it

```
import pandas as pd
df = pd.read_csv('data/fake_job_postings.csv')
print(df.shape)
print('Fraudulent:', df['fraudulent'].sum(), 'out of', len(df))
df.head()
```

Should print about (17880, 18) and Fraudulent: 866 out of 17880. About 5% fraudulent.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.

Class 1 — Data Cleaning

Scenario reminder: Carlos drops the CSV on your desk. Many columns have missing values. The location column is one big string. Your job today: clean and explore the imbalance.

Your goal

Look at missing values. Parse the location. Understand the imbalance.

Inputs

- data/fake_job_postings.csv

Outputs

- jobs_step1.parquet saved in Drive
- 3+ exploratory charts
- 1 chart for Carlos

Phase A — Explore the data first (15 min)

Exploratory chart 1 — Class imbalance

- **Question:** “How many fraudulent vs real?”
- **HINTS:**
 - `df['fraudulent'].value_counts().plot.bar()`
 - Add the percentage as text on the bar.
- **What you learn:** ~95% real, ~5% fraudulent. **Big imbalance.**

Exploratory chart 2 — Missing values per column

- **HINTS:** `df.isna().sum().sort_values(ascending=False).plot.barh()`
- **What you learn:** Some columns are missing >60% of the time. Examples: salary_range, department, benefits.

Exploratory chart 3 — Missing values vs fraudulent

- **HINTS:**
 - Count missing columns per row: `df['n_missing'] = df.isna().sum(axis=1).`
 - GroupBy fraudulent, mean of n_missing.
 - Bar chart.
 - **What you learn:** Fake postings tend to have MORE missing fields. (Scammers are lazy.)
-

Phase B — Clean the jobs table (45 min)

Step 1 — Load the CSV

- **HINTS:** `pd.read_csv('data/fake_job_postings.csv').`
- **EXPECTED:** ~17,880 rows × 18 columns.

Step 2 — Look at the DataFrame

- **WHAT:** Run `.info()`, `.head()`, `.shape`, `.dtypes`.
- **EXPECTED:** All text columns are object. The numeric columns `has_company_logo`, `has_questions`, `telecommuting`, `fraudulent` are all int 0/1.

Step 3 — Parse location

- **WHAT:** location is “US, NY, New York”. Comma-separated.
- **HINTS:**
 - Split by comma:
`loc_parts = df['location'].fillna('').str.split(',', expand=True)`
`df['country'] = loc_parts[0].str.strip()`
`df['state'] = loc_parts[1].str.strip() if loc_parts.shape[1] > 1 else None`
`df['city'] = loc_parts[2].str.strip() if loc_parts.shape[1] > 2 else None`

Step 4 — Count missing fields per row

- **WHAT:** This is a STRONG signal (scammers often leave fields blank).
- **HINTS:**
 - `df['missing_field_count'] = df.isna().sum(axis=1).`

Step 5 — Replace missing text fields with empty string

- **WHAT:** For each text column (`description`, `requirements`, `benefits`, `company_profile`, `title`).
- **HINTS:**
 - `df['description'] = df['description'].fillna('')`.
 - Repeat for the others.
- **WHY:** TF-IDF in M7 cannot handle NaN.

Step 6 — Decide what to do with salary_range

- **WHAT:** Mostly missing (~85%). Two options:
 - **A** Drop the column.
 - **B** Make a binary flag: `has_salary_listed = (salary_range.notna()).astype(int).`
- **YOUR CHOICE:** B is safer. The presence/absence of salary is itself a signal.

Step 7 — Write down what you did

In a markdown cell, list each decision.

Step 8 — Save to Drive

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/fake_jobs_lab/jobs_step1.parquet')`.
-

Phase C — Make ONE chart for Carlos (15 min)

Carlos's chart — “Fake postings have 30% more missing fields than real ones”

- **HINTS:**
 - GroupBy `fraudulent`, mean of `missing_field_count`.
 - Bar chart.
 - **Title:** “Missing field count by class — fake postings have 7.3 missing fields on average vs 5.6 for real.”
 - **Takeaway for Carlos:** “Auto-flag any posting with >7 missing fields. That alone catches ~30% of fakes with low false positives.”
-

Common mistakes in Class 1

Mistake	What goes wrong
Use accuracy to measure performance	95% baseline by always saying “real”. Useless. Use F1.
Drop rows with missing text columns	You lose data. Replace with ' ' instead.
Forget to parse location	Country/state/city info is wasted.

Self-check before Class 2

- ☐ Loaded 17,880 rows.
 - ☐ `location` is parsed into country, state, city.
 - ☐ `missing_field_count` exists.
 - ☐ Empty text fields are ' ' not NaN.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Carlos.
 - ☐ `jobs_step1.parquet` saved.
-

Class 2 — Encoding and Scaling

Scenario reminder: Carlos says: “The `industry` column has 130 different values. The `employment_type` has 5 (full-time, part-time, etc.). The `country` has 90+ values. Make these usable for the model.”

Your goal

Encode the categorical columns. Decide which to one-hot and which to target-encode.

Inputs

- `jobs_step1.parquet`

Outputs

- `jobs_step2.parquet` in Drive
-

Phase A — Explore the data first

Exploratory chart 1 — Top 20 industries

- **HINTS:** `value_counts().head(20).plot.barh()`.
- **What you learn:** IT and Marketing dominate. Many small industries.

Exploratory chart 2 — Fraud rate by industry

- **HINTS:**
 - Filter to industries with >50 postings.
 - GroupBy industry, mean of fraudulent.
 - Sort descending, head 10.
- **What you learn:** Some industries have much higher fraud rate (e.g., Marketing). Scammers favor them.

Exploratory chart 3 — Employment type distribution

- **HINTS:** `value_counts().plot.bar()`.
 - **What you learn:** Most are full-time. A few “Other” — suspicious.
-

Phase B — Encode and scale

Step 1 — One-hot encode `employment_type`

- **WHAT:** Only 5 values. Safe to one-hot.
- **HINTS:** `pd.get_dummies(df, columns=['employment_type'], prefix='emp')`.

Step 2 — One-hot encode `required_experience` and `required_education`

- **HINTS:** Same pattern. Both have ~5-7 values.

Step 3 — Target-encode `industry`

- **WHAT:** 130 industries. One-hot would add 130 columns.
- **HINTS:**
 - Compute mean fraud rate per industry.
 - WARNING: Only on TRAIN data in Class 4. For EDA: `df['industry_fraud_rate'] = df.groupby('industry')`

Step 4 — Target-encode `function`

- **HINTS:** Same as industry. ~40 unique values.

Step 5 — One-hot encode `country` (top 10 only)

- **WHAT:** Most postings are “US”. Keep top 10 + “Other”.
- **HINTS:**
 - Find top 10 countries.
 - For each: `df[f'country_{c}'] = (df['country'] == c).astype(int)`.

Step 6 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/fake_jobs_lab/jobs_step2.parquet')`.
-

Phase C — Make ONE chart for Carlos

Carlos’s chart — “Top 10 highest-risk industries”

A horizontal bar chart of the top 10 industries by fraud rate (with >50 postings filter).

- **HINTS:**
 - Filter, GroupBy, mean, sort, head 10.
 - Plot horizontally.
 - **Title:** “Industries with the highest fraud rates.”
 - **Takeaway for Carlos:** “Add extra manual review for these 10 industries. Quick win.”
-

Common mistakes in Class 2

Mistake	What goes wrong
Target-encode on FULL data	Leakage.
One-hot encode industry directly	130 new columns. Slow.
Drop missing industry rows	You lose data. Fill with “unknown” instead.

Self-check before Class 3

- ☐ `employment_type`, `required_experience`, `required_education` are one-hot.
 - ☐ `industry` and `function` are target-encoded.
 - ☐ Top-10 countries are one-hot.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Carlos.
 - ☐ `jobs_step2.parquet` saved.
-

Class 3 — Feature Engineering

Scenario reminder: Carlos says: “Now make me the SMART columns. Does the description contain a phone number? An email? A dollar sign with too many digits? These are scam signals.”

Your goal

Engineer signals from the text columns. Find fraud fingerprints.

Inputs

- `jobs_step2.parquet`

Outputs

- `jobs_step3.parquet` in Drive
-

Phase A — Explore the data first

Exploratory chart 1 — description_length by class

- After Step 1 below.
- **HINTS:** GroupBy fraudulent, mean of description_length, bar chart.
- **What you learn:** Fake descriptions tend to be SHORTER or strangely structured.

Exploratory chart 2 — has_phone_in_text by class

- After Step 3 below.
- **What you learn:** Real jobs RARELY put a phone in the description (they use the platform). Fake jobs often do (they want direct contact).

Exploratory chart 3 — n_exclamations_in_text by class

- **What you learn:** Fake postings often have many “!” — “GREAT OPPORTUNITY!!! APPLY NOW!!!”

Phase B — Engineer the features

Step 1 — Length features

For each text column (description, requirements, benefits, company_profile, title):

New column	How to make it
description_length	df['description'].str.len()
requirements_length	df['requirements'].str.len()
benefits_length	df['benefits'].str.len()
company_profile_length	df['company_profile'].str.len()
title_length	df['title'].str.len()

Step 2 — Title caps signals

New column	How to make it
n_caps_in_title	df['title'].str.findall(r'\b[A-Z]{2,}\b').apply(len)
title_all_caps_ratio	Ratio of CAPS letters to total letters in title

- **WHY:** “URGENT HIRING NOW!!!” all-caps = fraud signal.

Step 3 — In-text contact signals (combine all text columns first)

Combine all text into one big string per row:

```
df['all_text'] = (df['title'].fillna('') + ' ' +  
                  df['description'].fillna('') + ' ' +  
                  df['requirements'].fillna('') + ' ' +  
                  df['benefits'].fillna('') + ' ' +  
                  df['company_profile'].fillna(''))
```

Then:

New column	How
has_phone_in_text	regex for phone numbers (\d{3}[-.\s]?\d{3}[-.\s]?\d{4})
has_email_in_text	regex for emails (\S+@\S+.\S+)
has_url_in_text	regex for URLs (http\S+)
n_exclamations_in_text	df['all_text'].str.count('!')
n_dollar_signs	df['all_text'].str.count(r'\\$')
mentions_money_now	regex for "\$1000+ weekly", "earn money fast", etc.

- **WHY:** Each is a scam fingerprint.

Step 4 — Save

- **HINTS:** df.to_parquet('/content/drive/MyDrive/fake_jobs_lab/jobs_step3.parquet').

Phase C — Make ONE chart for Carlos

Carlos's chart — "Scam fingerprints"

A grouped bar chart showing the rate of each signal in real vs fake postings: - has_phone_in_text - has_email_in_text - n_exclamations_in_text > 3 - title_all_caps

- **HINTS:**
 - GroupBy fraudulent, mean of each binary signal.
 - Grouped bar chart.
- **Title:** "Scam fingerprints — fake postings are 8x more likely to include a phone in the text."
- **Takeaway for Carlos:** "Any 2+ of these signals together = block immediately."

Common mistakes in Class 3

Mistake	What goes wrong
Forget .fillna('') before .str.len()	Crashes on NaN.
Use a regex without \b boundary	Matches "EU" inside "BUREAU". False positives.
Compute keyword flags per column separately	Miss cross-column scams. Combine first.

Self-check before Class 4

- ☐ Length features for all 5 text columns.
- ☐ Title caps signals.
- ☐ In-text contact signals (phone, email, URL).
- ☐ Exclamation and dollar-sign counts.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Carlos.
- ☐ jobs_step3.parquet saved.

Class 4 — Feature Selection

Scenario reminder: Carlos says: "You have ~30 columns. Pick the best 12-15. Tell me which signals matter most."

Your goal

Pick the best 12-15 columns.

Inputs

- jobs_step3.parquet

Outputs

- jobs_step4.parquet in Drive
 - A short markdown report
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap

- **HINTS:** `sns.heatmap(df[numeric_cols].corr(), annot=True, fmt='.2f')`.

Exploratory chart 2 — Mutual information bars

- After Step 4 below.

Exploratory chart 3 — Random Forest importance bars

- After Step 5 below.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **HINTS:**
 - `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`.
 - **CRITICAL:** `stratify=y` because we have only 5% positives. Otherwise the test set might have very few fakes.

Step 2 — Variance threshold

- **HINTS:** `VarianceThreshold(threshold=0.01)`.

Step 3 — Correlation pruning

- **HINTS:** Drop pairs with `lcorr > 0.9`. Common candidates: lengths and word-counts of the same column.

Step 4 — Mutual information

- **HINTS:** `mutual_info_classif(X_train[numeric_cols], y_train)`.

Step 5 — Random Forest importance

- **HINTS:**
 - `RandomForestClassifier(n_estimators=100, max_depth=10, n_jobs=-1, class_weight='balanced')`.
 - `class_weight='balanced'` is critical here because of the imbalance.

Step 6 — Pick the final 12-15 columns

- **WRITE DOWN WHY** for each.
- **EXPECTED TOP FEATURES:**
 - missing_field_count, has_phone_in_text, industry_fraud_rate, has_company_logo, n_exclamations_in_text, description_length.

Step 7 — Save

- Keep only selected + fraudulent. Save as jobs_step4.parquet.
-

Phase C — Make ONE chart for Carlos

Carlos's chart — “Top 10 fraud predictors”

A horizontal bar chart.

- **Title:** “Top 10 predictors of fraud.”
 - **Takeaway:** “Missing fields, phone in text, and missing company logo are the top 3. Build the model around these.”
-

Common mistakes in Class 4

Mistake	What goes wrong
Forget stratify=y	Test set may have ZERO fakes. Useless test.
Use accuracy to validate	95% trivial baseline. Use F1.
Drop a column because it has 30% missing	Missing-ness IS the signal here.

Self-check before Class 5

- ☐ Train/test split with stratify=y done FIRST.
 - ☐ 12-15 columns + fraudulent.
 - ☐ WHY for each column written.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Carlos.
 - ☐ jobs_step4.parquet saved.
-

Class 5 — Pipelines

Scenario reminder: Carlos says: “Your code is in 4 notebooks. When a new posting arrives, will you copy 4 notebooks? No. We need ONE Pipeline. And remember: this is an IMBALANCED problem. Build for F1, not accuracy.”

Your goal

Build ONE Pipeline that handles the 5% imbalance correctly.

Inputs

- jobs_step4.parquet

Outputs

- `fake_jobs_pipeline.joblib` in Drive
-

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix

- After training.
- **HINTS:** `ConfusionMatrixDisplay.from_estimator(pipeline, X_test, y_test)`.

Exploratory chart 2 — Precision-Recall curve

- **WHAT:** With imbalanced data, PR curve is MORE useful than ROC curve.
- **HINTS:** `from sklearn.metrics import PrecisionRecallDisplay. PrecisionRecallDisplay.from_estimator(pipeline, X_test, y_test)`.

Exploratory chart 3 — ROC curve

- For completeness.
-

Phase B — Build the pipeline

Step 1 — Decide numeric vs categorical columns

- **EXAMPLE:**
 - numeric: `description_length, requirements_length, company_profile_length, title_length, missing_field_count, has_phone_in_text, has_email_in_text, n_exclamations_in_text, n_caps_in_title, industry_fraud_rate, function_fraud_rate`.
 - categorical: `country (one-hot top 10), employment_type (one-hot), required_experience (one-hot)`.

Step 2 — Numeric mini-pipeline

- **HINTS:**
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in: `'median'`.

Step 3 — Categorical mini-pipeline

- **HINTS:**
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in: `'most_frequent', 'ignore', False`.

Step 4 — ColumnTransformer

- **HINTS:**
 - `ColumnTransformer(transformers=[('num', ..., numeric_cols), ('cat', ..., categorical_cols)])`.

Step 5 — Add the model — IMBALANCE-AWARE

- **HINTS:**
 - `LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)`.
- **WHY `class_weight='balanced'`?** Without this, the model says “real” for everyone. 95% accuracy. ZERO catches. With this, the model cares equally about both classes.

Step 6 — Train and evaluate

- **HINTS:**
 - `full_pipeline.fit(X_train, y_train)`.
 - `from sklearn.metrics import classification_report, f1_score`.
 - `Print classification_report(y_test, y_pred)`.
 - Look at F1 on the FRAUD class (class 1).
- **EXPECTED:** F1 around 0.40-0.60. Recall on fraud class around 0.60-0.80. Precision lower (lots of false alarms).

Step 7 — Discuss the trade-off

- **WHAT:** If recall is too low, fake postings slip through. If precision is too low, you block too many real jobs.
- **HINTS:**
 - Try a few thresholds (e.g., 0.3, 0.5, 0.7) using `predict_proba`.
 - For each, print precision and recall on the fraud class.
- **WRITE DOWN:** Which threshold you would deploy and WHY.

Step 8 — Save the trained pipeline

- **HINTS:** `joblib.dump(full_pipeline, '/content/drive/MyDrive/fake_jobs_lab/fake_jobs_pipeline.jobl`

Phase C — Make ONE chart for Carlos

Carlos’s chart — “Precision vs Recall trade-off”

A line plot of precision vs recall as the threshold changes.

- **HINTS:**
 - Use `precision_recall_curve` from sklearn.
 - Plot precision on y-axis, recall on x-axis.
- **Title:** “At our chosen threshold (0.4), we catch 65% of fakes and wrongly block 8% of real jobs.”
- **Takeaway for Carlos:** “Each percentage point of recall = ~10 more fakes blocked per week. Each point of precision lost = ~50 more real jobs falsely flagged. We chose recall.”

Common mistakes in Class 5

Mistake	What goes wrong
Forget <code>class_weight='balanced'</code> Use accuracy in <code>classification_report</code>	Model says “real” for everyone. F1 on fraud = 0. Looks great (95%) but USELESS. Look at F1 on fraud class.

Mistake	What goes wrong
Use a default threshold of 0.5 without trade-off discussion	Skipped the most important step for an imbalanced problem.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model with `class_weight='balanced'`.
- ☐ F1 on fraud class above 0.40.
- ☐ You tested 3+ thresholds and chose one.
- ☐ Confusion matrix + PR curve + ROC curve charts.
- ☐ `fake_jobs_pipeline.joblib` saved.

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Carlos is in the meeting room. He wants the FINAL clean dataset in 90 minutes.

Your goal

Take the raw CSV. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Outputs

- `fake_jobs_clean.parquet` (~17,880 rows × 21 columns) in Drive
- `findings.md` (~1 page)
- Your Colab notebook

Phase A — Explore the data first (10 min)

Reuse 3 of your best charts: 1. Class imbalance (5% fraud). 2. Missing field count by class. 3. Top 10 fraud predictors.

Phase B — Build the final dataset (70 min)

Required output schema

#	Column	Type	Source
1	<code>job_id</code>	<code>int</code>	raw
2	<code>has_company_logo</code>	<code>int (0/1)</code>	raw
3	<code>has_questions</code>	<code>int (0/1)</code>	raw
4	<code>telecommuting</code>	<code>int (0/1)</code>	raw
5	<code>country</code>	<code>string</code>	parsed from location
6	<code>state</code>	<code>string</code>	parsed from location
7	<code>employment_type</code>	<code>string</code>	raw

#	Column	Type	Source
8	required_experience	string	raw
9	required_education	string	raw
10	industry_target_encoded	float	engineered (train-only)
11	function_target_encoded	float	engineered (train-only)
12	description_length	int	engineered
13	requirements_length	int	engineered
14	company_profile_length	int	engineered
15	title_length	int	engineered
16	n_caps_in_title	int	engineered
17	has_phone_in_text	int (0/1)	engineered
18	has_email_in_text	int (0/1)	engineered
19	n_exclamations_in_text	int	engineered
20	missing_field_count	int	engineered
21	fraudulent	int (0/1)	TARGET (raw)

(Keep the raw description text in a separate file for Module 7.)

Lab phases (timed)

Phase	Time	What you do
1. Load	10 min	Load CSV. Confirm 17,880 rows + 866 fraud.
2. Clean	15 min	Parse location. Fill empty text. Count missing per row.
3. Encode	15 min	Target-encode industry/function. One-hot top-10 country, employment_type, etc.
4. Engineer	20 min	Lengths, caps signals, contact signals (phone/email/URL), exclamation counts.
5. Validate + save	10 min	Check 21 columns + dtypes. Save .parquet.
6. Findings	10 min	Write findings.md.

Total: 90 minutes.

Phase C — Findings report for Carlos (10 min)

Write findings.md with these sections:

Final numbers

- Final row count: _____
- Fraud rate: _____% (should be ~5%)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

• _____

One chart that summarizes everything

Embed your most important chart.

Grading rubric (100 points)

Item	Points
All 21 columns exist with right names and dtypes	25
Cleaning decisions in markdown	10
Pipeline reproducible (one command, raw CSV to .parquet)	15
Target encoding on TRAIN ONLY	10
class_weight='balanced' used in Class 5 baseline	5
Threshold trade-off documented	5
At least 3 exploratory + 1 explanatory chart per class	15
findings.md discusses imbalance and trade-offs (not just numbers)	10
Code is clean	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - fake_jobs_clean.parquet - Your Colab notebook - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.**
2. **Save your notebook to Drive often.**
3. **Save intermediate .parquet files to Drive.**
4. **Pair-program.** Switch every 20 minutes.
5. **Accuracy is a TRAP.** Use F1. Memorize this.
6. **Missing-ness IS a signal.** Do not drop columns just because they have many missing values.
7. **Test multiple thresholds.** Default 0.5 is rarely the best.
8. **Ask the mentor early.** If stuck for 20 minutes, ask.

Good luck.